ROM CRC, CPU clock measurement, FPU probe, RTC/NVRAM, an 18.2 Hz tick, and a complete INT 14h serial driver.

The operating-system-facing half does not exist yet. There is no bootstrap — `_main_()` ends by calling `testmain()` and then powers the CPU down. INT 19h, INT 10h, INT 16h, and INT 17h all fall through `stub.asm` into a shared "invalid function" return that sets carry and `AH=01`. INT 13h exists as scaffolding but has never been executed successfully — it contains at least five independent bugs, any one of which is fatal, and the POST never populates the drive tables it depends on.

Nothing here is architecturally wrong; it is unfinished, in a predictable order. About 52K of the 64K ROM window is still free, which is more than enough room for everything below.

ROM IMAGE TODAY	WRITABLE DATA SEGMENT	INT VECTORS THAT WORK	BLOCKING DEFECTS
12,896 B	**0 B**	**4**	**11**
of 65,536 — 80% free	_DATA + _BSS are empty	11h, 12h, 14h, 1Ah (+ IRQ0)	across 13h_disk, diskide, main

---

01

What already works

Read against the source, POST is substantially complete and the low-level drivers are real code, not placeholders.

Board initialisation — `start.asm`

- Expanded I/O unlock (the `REMAPCFG` state machine), flags-register sanity test, then table-driven word and byte port init.
- Chip selects: UCS = 64K ROM at `F0000` (8-bit, 5 ws); CS0 = ECB I/O at `0400–04FF`; CS1 = IDE at `01F0` (8-bit, 3 ws); CS2 = DRAM `00000–FFFFF` (16-bit, 2 ws); CS3 = external memory at `B0000`; CS4 = SRAM, *disabled* at `HALF_MEM=0`. Refresh unit and watchdog bus monitor both enabled.
- Memory: walking-ones dword march over `0000:0000–9000:FFFF`, plus SRAM sizing. Extended memory is sized by dropping into 32-bit protected mode (`sizer.asm`) and returning to real mode — that machinery is working and will be reusable for INT 15h AH=87h.
- ROM CRC16 verified against the value planted at `0xFFEE` by `bin2hex`.
- CPU clock measured with the Timer 0 / Timer 1 gate trick, then PSCLK reprogrammed to exactly 1 MHz and the refresh counter recalculated for the measured clock. Nicely done.
- FPU probed via `FNINIT` / `FNSTSW` / `FNSTCW`; `equip_flag` bit 1 set accordingly.

Drivers and services

- **INT 14h** (`14h_sio0.asm`) — complete and correct: init, write, read, status, PS/2 extended init, divisor table to 460,800 baud. This is the console the whole BIOS currently speaks through.
- **INT 1Ah** (`1Ah_time.asm`) — functions 00–05 plus a DS1302-specific 20h–24h group (burst NVRAM read/write, write-protect, trickle-charge control). Timer 0 drives INT 08h at 18.2066 Hz, chaining INT 1Ch and servicing the BDA tick count, FDC motor timeout and the printer/serial countdown bytes.
- **INT 11h / INT 12h** — real, reading `equip_flag` and `memory_size` from the BDA.
- **ICU** — master and slave 8259s initialised AT-compatibly (vectors 08–0F and 70–77), with `mask_interrupt` / `unmask_interrupt` helpers.
- **IDE** (`diskide.asm`) — 8-bit PIO read, write, IDENTIFY and init. It enables ATA `SET FEATURES 01h` (8-bit transfers), which the board requires because CS1 is wired 8-bit.
- **C runtime** — `printf` over INT 14h, `getchar` / `getline` , and `option_get` (a numbered-choice prompt). These are assembled into `set_top()` , a genuinely interactive SETUP menu reached by pressing a key during the POST scroll, backed by a CRC-protected 31-byte NVRAM image. Two of its four entries — Fixed Disks and Floppy Disks — are `printf("...not implemented")` stubs.

THERE IS NO COMMAND MONITOR

`testmain()` is a straight-line test *script*, not a monitor: what it runs is fixed at compile time by the `#define` block at the top of `testmain.c` , and it returns when it reaches the end. There is no prompt and no command dispatch anywhere in the ROM. The pieces one would be built from all exist and work — the console, the line editor, the menu loop — so Phase 0 below assembles them into one, and hangs it off the `set_fixed()` stub that is already in the SETUP tree.

02

The interrupt map

This is the clearest picture of where the BIOS stands. Note the block of entries that share one fall-through in `stub.asm`: they do not `IRET` quietly — they return `CF=1, AH=01`, which is what DOS will see when it tries to print a character or read a key.

VECTOR	SERVICE	STATE	NOTES
00–07	CPU exceptions	STUB	All alias to a single `IRET`.
08	IRQ0 — timer tick	WORKING	18.2066 Hz, chains INT 1Ch, maintains BDA counters.
09–0F	IRQ1–7	EOI ONLY	Non-specific EOI then `IRET`. Fine until you use them.
10	Video	RETURNS ERROR	Falls into the invalid-command return. Every DOS console write fails.
11	Equipment list	WORKING	Video and floppy bits are never set, though.
12	Conventional memory	WORKING	Reports 640K.
13	Disk	NON-FUNCTIONAL	Only 41h/42h are written, and both fail. See section 03.
14	Serial	COMPLETE	The strongest module in the tree.
15	Misc / system	PARTIAL	Only AH=86h (delays ≥250 ms) and a 4Fh stub. 87h/88h/C0h/C1h missing.
16	Keyboard	RETURNS ERROR	COMMAND.COM cannot read input at all.

17	Printer	RETURNS ERROR	Should report "not present" cleanly instead.
18	ROM BASIC / boot failure	RETURNS ERROR	Should land in the monitor.
19	Bootstrap loader	MISSING	Nothing in the image ever attempts a boot.
1A	Time / RTC	WORKING	Plus the DS1302 extension group.
1B / 1C	Break / user tick	IRET	Correct as-is.
1D / 1E / 1F	Parameter tables	NOT TABLES	Vectors point at code, not at the tables DOS may read.
40	Floppy	INVALID CMD	Correct — there is no FDC on the board.
41 / 46	Fixed disk parameter tables	NOT TABLES	Both point at an `IRET` instruction. INT 13h dereferences them as `T_DISKTAB`.
70–77	IRQ8–15	EOI ONLY	Cascade EOI handled correctly.

Source: page0vectors in start.asm, cross-referenced against stub.asm fall-through order.

---

03 Defects found by inspection

These are code-reading findings, not observed failures — confirm each on hardware as you go. Taken together they say the same thing: **the INT 13h path has never successfully executed.** That is consistent with the symptom you describe.

01 `main.c — if (code) bda.disk_tab[0] = FX_IDEm;` BLOCKER

Drives are registered only when `code` is non-zero — that is, only when the NVRAM

checksum is *bad*, the clock is stopped, or charging is disabled. On a healthy board no disk is ever registered. The condition is inverted.

02 `POST — bda.hd_number is never written` BLOCKER

Nothing in the image assigns it, so it stays 0. `get_disk_table` compares the drive number against it and rejects every call, including 41h and 42h. There is no disk enumeration step at all.

03 `13h_disk.asm — get_disk_table , mov di,[di+dtab_vectors]` BLOCKER

`dtab_vectors` is a table of *bytes* (`db 0x41,0x46,...`) but is read as a word. Index 0 yields `0x4641` instead of `0x41` , so the subsequent `shl di,2 / les di,[di]` loads a garbage far pointer. Needs a byte load, zero-extended.

04 `13h_disk.asm — fn41_check_extensions_present` BLOCKER

After `get_disk_table` returns the table in `ES:DI` , the flags test uses `[bx+disk_flags]` . `BX` at that point is still the caller's `0x55AA` magic number. Should be `[di+disk_flags]` .

05 `13h_disk.asm — fn42_read_sector , mov bx,1` BLOCKER

`read_tab` is `{ microSD=0, DualSD=0, IDE_READ_SECTOR }` . With `bx=1` , `add bx,bx` gives byte offset 2 — word index 1 — which is the *null* DualSD entry. The call target must be derived from `disk_tab[]` , not hard-coded; for IDE it needs to select index 2.

06 `13h_disk.asm — floppy_call , ret 2` BUG

Inside an interrupt handler this is a *near* return: it pops IP, discards CS, and leaves the caller's flags on the stack. Must be `retf 2` . Latent today because there is no floppy, but it will bite the moment something calls INT 13h with `DL<0x80` — which DOS does while probing.

07 `13h_disk.asm — CHS entry points` BLOCKER

Functions 00, 02, 03, 04, 08, 15 and packet calls 43, 44, 47, 48 are all aliased to

`ret_invalid_command`. AH=02h (CHS read) is what a DOS boot sector and IO.SYS actually use; without it nothing loads.

08 `diskide.asm — IDE_READ_SECTOR / IDE_WRITE_SECTOR` BLOCKER

DRQ is polled once *before* the sector loop. ATA raises DRQ per sector during multi-sector PIO, so any transfer of more than one sector will run ahead of the drive and desynchronise. DOS reads multiple sectors constantly. The DRQ wait belongs inside the loop.

09 `diskide.asm — error handling` BUG

The ERR bit and the error register are never read, so every failure collapses to `AX=-1`. INT 13h cannot produce meaningful status codes, and DOS's retry logic keys off them.

10 `diskide.asm — timeouts` BUG

`ide_wait_not_busy` and `ide_wait_drq` spin on `CX=0FFFFh` with `loopnz`. That is both clock-dependent (16–33 MHz) and far too short for drive spin-up or a reset. Use the 1 MHz Timer 1 that POST already sets up.

11 `main.c — end of _main_()` BLOCKER

`testmain(); printf("\nShutdown.\n"); return 8;` — control returns to `exit_` in start.asm, which enters PWRCON power-down and halts. There is no `int 19h` anywhere in the ROM.

---

04 Constraints that shape the work

Four properties of this build will dictate how you write every new module. Two of them are easy to violate accidentally.

THE ONE THAT WILL CATCH YOU OUT

There is no writable data segment. The makefile puts `_TEXT`, `CONST`, `CONST2`, `_DATA` and `_BSS` all in `DGROUP`, which lives entirely in ROM — `start.map` shows `_DATA` and `_BSS` at zero length. Any C global or `static` you add lands in ROM. `const` data is fine and belongs there — `testmain.c` already uses `static const` tables — but anything you intend to *write* will silently discard the write. All mutable state must live in the BDA at `0040:xxxx` or on the stack. Add the field to `bda.h` and let `copt` regenerate `bda.inc`.

- **64K ROM window, ~52K free.** The UCS covers `F0000–FFFFF` only, and the current image is 12,896 bytes. Everything below fits comfortably; you are not space-constrained.
- **The console is SIO0, and only SIO0.** There is no video controller and no PS/2 keyboard on the board. INT 10h and INT 16h must be a terminal emulation layered on the serial port — that is the single largest piece of new code in this plan, and the one most likely to need iteration.
- **IDE is wired 8-bit.** CS1 is configured with the bus-size bit clear, which is why `IDE_INITIALIZE` issues `SET FEATURES 01h`. Only drives that honour the 8-bit transfer feature will work. CompactFlash cards do; most spinning IDE drives do not. Plan on CF.
- **The POST stack sits at `A800:0000`.** Because CS4 is disabled at `HALF_MEM=0`, that address is actually served by DRAM through CS2. It is above the 640K the BIOS reports, so it will not collide with DOS — worth keeping that way deliberately rather than by accident.
- **BIOS code executes from 8-bit ROM at 5 wait states.** Every INT 13h and INT 10h call pays for that. If disk or console throughput disappoints later, `SHADOW_MODE` is the lever.

05

The plan

Ordered by dependency, not by size. Phase 0 builds the instrument the rest of the plan is tested with; disk comes next, because with that instrument in hand you can prove out storage over the serial console you already have, before taking on the much fuzzier console emulation. Effort figures assume you already know this codebase.

PHASE 0 Ground truth and a monitor ~1 day

> Before changing anything, make the build reproducible and build the instrument you will diagnose everything else with. You are about to do a lot of surgery on code that has never run, and right now the ROM has no way to ask the hardware a question interactively.

`git` Put the tree under version control. There is no repository here today, and the next four phases touch nearly every file.

`build` Fix the `rom128/256/512.hex` targets — they currently emit empty files (`:00000001FF`); only `rom064.hex` is real. Generate `date.inc` from the build rather than hand-editing it.

`new` **Build the command monitor.** A `while` loop over `getline()` and a small command table is all that is missing — `printf` , `getchar` , `getline` and `option_get` already work. Hang it on the `set_fixed()` entry in `set_top()` , which is a stub today and is exactly where disk commands belong.

`commands` Start with four: IDENTIFY dump, LBA sector dump in hex, BDA dump, and an I/O port peek/poke. These are the instrument for all of Phase 1 and the first six rungs of the bring-up ladder.

`watch out` No writable statics — see section 04. A command table must be `const` ; any line buffer or parsed argument has to be a local on the stack.

`log` Capture a serial log of the current POST as a baseline to diff against.

PHASE 1 **Make INT 13h real** 2–3 days

> The deepest work in the plan. Target: `AH=02h` CHS reads and `AH=08h` geometry queries that a DOS boot sector will accept, backed by an IDE driver that survives multi-sector transfers.

`new file` **Disk enumeration at POST.** Add a `hdinit` module: soft-reset the interface, issue IDENTIFY to master and slave, and parse words 1/3/6 (default geometry), 49 (capabilities) and 60–61 (LBA sector count).

`tables` Build a `T_DISKTAB` per drive and point the **INT 41h and INT 46h vectors** at them — they currently point at an `IRET`. Set `bda.hd_number` and `bda.disk_tab[]` here, unconditionally, and delete the inverted assignment in `main.c`.

`geometry` Choose an LBA-assist translation (16/32/64/128/255 heads × 63 sectors) so the reported cylinder count stays ≤1024. Store both the physical and translated geometry; AH=08h returns the translated one.

`port` **Port the CHS entry points from `SBC386/HardDisk/DIDE.ASM`** — the SBC-188 driver already implements fn00, 02, 03, 04, 08, 15 and a working `cv_lba` CHS→LBA conversion against the same BDA layout. This is the single biggest shortcut available to you.

`fix` Repair defects 03–07: byte-load `dtab_vectors`, use `DI` in fn41, derive the driver index from `disk_tab[]` in fn42, and change `floppy_call` to `retf 2`.

`driver` In `diskide.asm`: move the DRQ wait inside the sector loop, read the error register on failure and map it to BIOS status codes, and replace the `loopnz` timeouts with 1 MHz Timer 1 deadlines.

`status` Record the last operation's result in `bda.hd_status` and implement fn01 (get status) — DOS reads it after failures.

PHASE 2 ## Bootstrap and hand-off ~1 day

> Small, and it makes the machine feel like a PC for the first time. Prove it with a hand-written boot sector that prints through INT 14h — that isolates the boot path from the console work in Phase 3.

`new file` **INT 19h.** Model it on `ATBIOS/test6.asm : BOOT_STRAP_1`, which you already have: clear `0000:7C00`, retry loop, read drive 80h cylinder 0 / head 0 / sector 1 via `AH=02h`, verify the `0AA55h` signature at offset 510, set `DL=80h`, then `jmp 0000:7C00` with DS=ES=0 and a sane SS:SP.

`POST` Replace the tail of `_main_()`: after the summary, issue `int 19h` instead of `testmain()`. Keep the self-test reachable from a keypress or a SETUP menu entry so you do not lose it.

`INT 18h` On boot failure, print a real message and drop into the Phase 0 monitor rather than halting or returning an error code. This is what makes a failed boot debuggable instead of silent.

`setup` Implement `set_fixed()` — currently it prints "not implemented". A boot-device byte in NVRAM plus the drive table it already has room for.

PHASE 3 ## Console — INT 10h, 16h, 17h 3–5 days, and the most likely to overrun

> A serial terminal pretending to be a PC display and keyboard. Scope this to what IO.SYS and COMMAND.COM actually call; resist building a full VGA BIOS.

`INT 10h` Minimum viable set: `0Eh` teletype, `0Fh` get mode, `03h` get cursor, `02h` set cursor, `06h/07h` scroll, `09h/0Ah` write char (+attr), `00h` set mode, `08h` read char/attr, `01h` cursor shape (accept and ignore), `05h` set page. Emit ANSI/VT100 sequences for positioning and scrolling; maintain `vid_mode`, `vid_columns` and `vid_cursor[]` in the BDA as you go.

`decision` Keep a real 80×25×2 text buffer at `B800:0000` (DRAM already covers it) and mirror writes to the serial line. It costs 4K of DRAM you are not using, makes `08h` / `09h` / `0Ah` honest, and gives programs that write video memory directly somewhere valid to land.

`INT 16h` Functions 00/01/02 and 10h/11h/12h, fed from a ring buffer in the BDA — `buffer_head`, `buffer_tail` and `kbd_buffer[16]` are already defined. Translate inbound VT100 escapes (arrows, Home/End, function keys) into PC scan codes, map DEL/BS sensibly, and synthesise a scan code for each ASCII character from a table.

`important` **Make serial receive interrupt-driven.** INT 14h is polled; if INT 16h only polls, every keystroke typed while DOS is producing output is lost. Enable the SIO0 receive interrupt and fill the BDA ring buffer from the handler. Decide this now — retrofitting it later means rewriting INT 16h.

`INT 17h` Return a clean "not present / timeout" status so DOS's LPT probe terminates instead of reading the invalid-command return.

`plumbing` Point `VIDEO_putchar_` in `stub.asm` at INT 10h so POST messages and DOS output share one path. Set the video bits (4:5) and floppy bits in `equip_flag`.

PHASE 4 INT 15h and the parameter tables

~1 day

> The remaining calls DOS and its standard drivers make. None are needed for a bare boot; all are needed before the system feels finished.

`AH=88h` Return `bda.extended_memory`. HIMEM.SYS will not load without it.

`AH=87h` Extended-memory block move. `sizer.asm` already contains working protected-mode entry and exit — lift that machinery rather than writing it again.

`AH=86h` Handle delays under 250 ms; the current code rejects them. Use the 1 MHz Timer 1.

`AH=C0h` Return a system configuration table (model FCh, submodel, BIOS revision, feature bytes).

Cheap, and several utilities query it.

`tables` Point INT 1Eh at a real disk base table. Verify the INT 41h/46h tables from Phase 1 are what DOS expects to find.

PHASE 5 ### Hardening — as needed

> After DOS boots. None of this blocks the milestone.

`reset` Ctrl-Alt-Del path: warm-boot flag `1234h` at `40:72` and a clean re-entry to INT 19h.

`speed` Shadow the ROM into DRAM (`SHADOW_MODE`) — the BIOS currently executes from 8-bit ROM at 5 wait states, which every disk and console call pays for.

`watchdog` Confirm the bus-monitor watchdog does not trip during long DOS I/O. It is enabled today (`WATCH_BUS 1`).

`slave` Second IDE device, and the SD-card driver slots already reserved in `read_tab` / `write_tab` .

---

06 # Bring-up ladder

Each rung is independently observable on the serial console, and each one only depends on the rungs below it. Do not skip ahead — a failure at rung 9 with rungs 1–8 unverified is very hard to diagnose.

01 **IDENTIFY dump**

Words

1/3/6 and 60–61 from the monitor. Proves the CF card, the 8-bit feature negotiation, and CS1 timing.

---

02 Raw LBA 0 dump

512 bytes in hex, ending in `55 AA`. Proves the PIO read

path.

---

03 Multi-sector read

Read 8 sectors in one command and compare against 8 single-sector reads. This is the test that catches defect 08.

---

04 INT 13h AH=08h

Geometry from the monitor. Proves

Proves enumeration, the disk tables, and the INT 41h vector.

---

05 **INT 13h AH=02h**

CHS read of cylinder 0 / head 0 / sector 1, byte-identical to rung 2. Proves the CHS→LBA conversion.

---

06 **Write and read back**

A scratch sector well away from anything you care about.

---

07 Custom boot sector, INT 14h output

A hand-written 512-byte sector that prints "HELLO" through the serial driver. Proves INT 19h with the console

still out of the picture.

---

08 Custom boot sector, INT 10h output

Same sector, printing via `AH=0Eh`. Proves the video emulation's most-used entry point. Then a variant that echoes INT 16h input.

---

09 DOS boot sector

A

real VBR that loads IO.SYS. Expect to iterate on INT 10h coverage here.

---

10 Full boot to the prompt

COMMAND.COM will exercise far more of INT 10h and INT 16h than anything before it.

Media and DOS version

Prepare a CompactFlash card on a PC: one primary FAT16 partition, **504 MB or smaller**, geometry 16 heads × 63 sectors. That keeps the whole boot chain inside plain CHS addressing with no 1024-cylinder translation, which removes an entire class of problem from the first bring-up. You can relax it later once the LBA packet calls are trusted.

Start with **MS-DOS 6.22**. Its BIOS demands are the most predictable and best documented, and the AT BIOS listing you already have in `SBC386/ATBIOS` is the exact reference for what it expects. Keep FreeDOS as a second target — it is more tolerant and more talkative about what it finds, which makes it a useful diagnostic when 6.22 fails silently.

---

07 Risks worth naming now

- **Serial input loss** is the most likely source of mysterious behaviour once DOS runs. Polled receive plus a busy CPU equals dropped keystrokes. Handle it in Phase 3, not later.
- **The ROM CRC check halts the machine.** POST verifies CRC16 against the value `bin2hex` plants at `0xFFEE` and executes `HLT` on mismatch. Any hand-patched image will brick silently — always go back through the makefile.
- **CS2 and UCS overlap at `F0000`.** Intel warns against overlapping chip selects and the README records real trouble here. It appears resolved, but a DOS program writing into segment F000 will land in DRAM, not ROM.
- **8-bit IDE limits your drive choice.** If a drive does not accept `SET FEATURES 01h`, no amount of BIOS work will make it read. Confirm at rung 1.

- **The 64K ROM window is a hard ceiling.** You have ample room today, but INT 10h with a text buffer, INT 16h with scan-code translation, and full INT 13h will together add real bulk. Watch `start.map` as you go.

Where the leverage is

Two pieces of code already in this tree do most of the heavy lifting if you reuse rather than rewrite: `SBC386/HardDisk/DIDE.ASM` is a complete, working INT 13h for the SBC-188 against the same BDA layout, and `SBC386/ATBIOS/ATBIOS/` is the IBM AT BIOS listing — the definitive answer to "what exactly does DOS expect here" for the bootstrap, the disk calls and the parameter tables. Between them, Phases 1 and 2 are largely a porting exercise. Phase 3 is the part that is genuinely new work.

08

Source tree inventory

Every source file in `SBC386/bios`, and whether the build actually touches it. Membership was determined from the makefile's `OBJECTS` list cross-checked against the module list in `start.map`, so "linked" means the object is genuinely in the ROM image, not merely that a rule exists for it.

40 of the 93 source files never reach the ROM. Thirty-eight are not compiled at all; two more are compiled into `sbc386.lib` but the linker never pulls them in. Almost all of it is either SBC-188 heritage, prototype-era experiments, or the author's own numbered backup snapshots.

SOURCE FILES	LINKED INTO THE ROM	INCLUDES & BUILD INPUTS	NOT IN THE BUILD
93	22	27	40

86 in root, 7 in lib/	19 asm/C + 3 from lib	7 of them generated	safe to remove after Phase 0

A · ASSEMBLY MODULES IN THE ROM

FILE	STATE	WHAT IT DOES
`start.asm`	LINKED	POST. Chip-select and port init tables, IVT population, memory march test, ROM CRC, CPU clock measurement, FPU probe, LED codes, then calls `_main_()` .
`boot.asm`	LINKED	The 16-byte reset block at `FFFF:0` — `jmp F000:0000` plus the date stamp. Both `%include` d by start.asm and assembled standalone to `boot.tmp` for exe2rom.
`sizer.asm`	LINKED	GDT prototype and the protected-mode entry/exit used to size extended memory in 1 MB steps. Reuse this for INT 15h AH=87h.
`crc.asm`	LINKED	CRC16 over `ES:BX` ; built as `crc16.o` with `-dCRC16=1` . Used for the ROM self-check and the NVRAM checksum.
`diskide.asm`	LINKED	8-bit PIO IDE driver: read, write, IDENTIFY, and the SET FEATURES 01h init. Defects 08–10 live here.
`13h_disk.asm`	LINKED	INT 13h dispatcher, driver tables and packet validation. Defects 03–07 live here.
`14h_sio0.asm`	LINKED	INT 14h serial driver plus `install_SIO0()` and the baud divisor table.
`15h_misc.asm`	LINKED	INT 15h dispatcher. Only AH=86h is written, and only for delays over 250 ms.
`1Ah_time.asm`	LINKED	INT 1Ah, the IRQ0 tick handler, and the entire DS1302 RTC/NVRAM bit-banger. The largest module in the tree.
`11h_12h.asm`	LINKED	INT 11h equipment word and INT 12h conventional memory size. Both trivial, both correct.
`icu.asm`	LINKED	`mask_interrupt` / `unmask_interrupt` for the 8259 pair, callable from C.
`errno.asm`	LINKED	Returns a far pointer to the `errno` word in the BDA — the C library's only writable global

errno.asm	LINKED	Returns a far pointer to the `errno` word in the BDA — the C library's only writable global.
stub.asm	LINKED	Placeholder handlers for every unimplemented vector. This is the file you delete from as Phases 2–4 land.

B · C MODULES IN THE ROM

FILE	STATE	WHAT IT DOES
main.c	LINKED	`_main_()` — installs the serial console, prints the banner and machine summary, enters SETUP when needed, then calls `testmain()`. Defects 01 and 11 live here.
set1302.c	LINKED	The SETUP menu: `set_top`, `option_get`, and the clock, date, charger and serial editors. `set_fixed()` and `set_floppy()` are stubs.
testmain.c	LINKED	The compile-time-selected test script — CPRINTF, day-of-week, tick display, and probes for 4UART / VGA3 / CVDU ECB boards.
getline.c	LINKED	Line editor over the serial console. One of the pieces the Phase 0 monitor is built from.
signon.c	LINKED	`pr_lic()` copyright and GPL banner, plus `get_time` / `get_date` wrappers over INT 1Ah.
strtobcd.c	LINKED	BCD string parser used by the SETUP date and time editors.

C · LIB/SBC386.LIB

FILE	STATE	WHAT IT DOES
lib/cprintf.c	LINKED	`printf` / `cprintf`. Output goes out through `VIDEO_putchar` in stub.asm, which today calls INT 14h directly.
lib/strlen.c	LINKED	Pulled in by cprintf's `%s` handling.
lib/uart_det.asm	LINKED	UART type detection. Reaches the ROM only because `test_4uart()` in testmain.c calls it.

`lib/atoi.c`	UNLINKED	In the library, but nothing references it.
`lib/testide.c`	UNLINKED	IDE exerciser. In the library, but `T_IDE` is 0 in testmain.c so it is never called. Worth reading before you write the Phase 0 monitor's disk commands.
`lib/libc.c`	UNUSED	Commented out of the library makefile's `OBJECTS` .
`lib/makefile`	BUILD	Builds `SBC386.lib` . Note it is *not* invoked by the top-level makefile — the library is stale unless you build it by hand.

D · INCLUDES, HEADERS AND GENERATED FILES

FILE	STATE	WHAT IT DOES
`seg_def.inc`	INCLUDED	Segment and DGROUP definitions. This file is the origin of the no-writable-data constraint in section 04.
`i386ex.inc`	INCLUDED	386EX peripheral register addresses and bit names, for assembly.
`i386ex.h`	INCLUDED	The same register set for C. Used by main.c.
`macro.inc`	INCLUDED	`binit` / `winit` init-table macros, `pushm` / `popm` , `get_bda` , and the GDT descriptor builders.
`timer.inc`	INCLUDED	Timer and PSCLK constants, the MS18 / MS50 divisors, and the baud rate base.
`serial.inc`	INCLUDED	Serial constants; included only by 14h_sio0.asm.
`CRC16TAB.INC`	INCLUDED	CRC16 lookup table for crc.asm.
`date.inc`	INCLUDED	The 8-character build date stamped into the reset block. Hand-edited — currently reads `09/26/26` . Phase 0 automates it.
		The BIOS Data Area layout, and the [illegible] rules that translate a C struct into

bda.h + bda.rul	BUILD INPUT	The BIOS Data Area layout, and the `copt` rules that translate a C struct into NASM equates. `bda.rul` drives all four generated `.inc` files.
bda.inc	GENERATED	From bda.h. Included by eleven modules — the shared vocabulary of the whole BIOS.
disktab.h → disktab.inc	GENERATED	Fixed-disk parameter table and the INT 13h packet structures. Central to Phase 1.
error.h → error.inc	GENERATED	BIOS status/error codes.
stack.h → stack.inc	GENERATED	`offset_ax`, `offset_flags` and friends — for reaching into a saved register frame from inside an interrupt handler.
nvram.h	INCLUDED	NVRAM layout and the device-code enum (`FX_IDEm`, `FX_uSD` …). Also pulls in bda.h and serial.h.
disk.h	INCLUDED	Disk driver signatures and the device-number enum. Included by set1302.c.
mytypes.h, cprintf.h, getline.h, ascii.h, serial.h, strtobcd.h	INCLUDED	Small C headers — base types, the printf and line-editor prototypes, ASCII names, serial config struct, BCD parser prototype.

E · NOT IN THE BUILD — THE DUST

FILE	STATE	WHAT IT IS, AND WHETHER TO KEEP IT
alloc.asm	UNUSED	EBDA / UMB allocator. `main.c` declares `ebda_alloc()` but never calls it. **Keep** — Phase 4 may want it.
baseio.asm	BROKEN	Dual-SD-card board I/O. It `%include`s `sdcard.inc`, which does not exist anywhere in the tree, so it cannot assemble even if you added it to the makefile.
microsd.c	UNUSED	On-board micro-SD driver skeleton. The matching slots in

		`read_tab` / `write_tab` in 13h_disk.asm are null pointers.
`ds1302.asm`	SUPERSEDED	Earlier standalone RTC driver. The live version lives inside 1Ah_time.asm.
`dsreg.asm`	SUPERSEDED	DS1302 register definitions in *MASM* syntax (`TITLE` , `PAGE` , `.xlist`) — wrong assembler for this build entirely.
`muldiv.asm`	UNUSED	Its entire body is wrapped in `%if 0` .
`notice.asm`	UNUSED	A boilerplate copyright block meant for inclusion everywhere. Nothing includes it.
`equates.asm`	UNUSED	SBC-188 equates.
`sbc188.h, libc.h`	UNUSED	SBC-188 leftovers. Both are still listed in the makefile's `CINCL` variable but included by nothing.
`pm_sio.asm, pm_test.asm, rm_test.asm`	UNUSED	Prototype-era protected-mode serial and memory-test experiments. `sizer.asm` is the survivor of this line of work.
`startup.asm`	SUPERSEDED	The old monolithic TEST1 ROM. The makefile still carries a rule for it, explicitly labelled "obsolete".
`seg.c, seg_at.asm, make.seg`	UNUSED	A self-contained three-file experiment proving `SEGMENT AT 40h` addressing of the BDA. Its conclusion is already baked into bda.inc.
`zero.asm, zero.inc, zero.h, zero.rul`	UNUSED	A BDA-zeroing helper, fully wired into the makefile — `zero.inc` is even listed in `INCLUDES` — but `%include` d by nothing. The corresponding zeroing code in start.asm is inside a `%if 0` .
`disk.inc`	UNUSED	A copt-generated copy of disk.h that no module includes.
`remover.h`	UNUSED	Three `#define` tricks that let one file serve as both a `.inc` and a `.h` . Nothing uses it, but it documents the technique — **keep the comment** if you delete the file.

`test.c, tcrc.c, testide.c (root)`	UNUSED	Scratch and host-side test programs. The root `testide.c` is a duplicate of `lib/testide.c`; check which is newer before deleting either.
`start.lkk`	UNUSED	An old wlink response file. The makefile line that used it (`WLINK @start.lkk`) is commented out.
`makefile.188, makefile.abs`	UNUSED	The SBC-188 makefile and an absolute-address build variant.
`1Ah_time.as0, diskide.as0, start.as1, startup.as0, uart_det.as0/.as1/.as2, main.c7`	BACKUPS	The author's own numbered snapshots of files that still exist in current form. `main.c7` is a Microsoft C7 variant of main.c. **Delete these only after Phase 0 puts the tree under git** — they are the sole record of some earlier revisions.

F · DOCUMENTATION

FILE	STATE	WHAT IT IS
`0UPDATE.TXT`	READ THIS	Mandatory board wiring updates, including the DCTL16-7 GAL change and the FPU-detect jumper. Check it before flashing anything.
`0README.TXT`	KEEP	Dated build history back to 2017 — the overlapping chip-select saga and the MF/PIC console fallback are both documented here.
`0USAGE.TXT`	KEEP	The BDA fields regrouped by subsystem rather than by address. Derived from bda.h; genuinely useful reference for Phases 1–3.
`COPYING`	KEEP	GPLv3.

Two makefile cleanups worth doing at the same time

The bottom of the makefile carries a "Leftovers from SBC188" dependency block naming eleven files that do not exist in this tree — `sio.c`, `nvram.c`, `debug.c`, `kbd.c`,